



Name:- Jaykishan J Saharan

Div:- D Roll no.:- 26

Course/Program:- SY BTech CSE

Subject:- DSA

DSA lab assignment 6.

09/12

* Title: write a c++ Program to demonstrate enqueue and display operations performed on simple and circular queue.

Q.1) What is queue data structure? Enlist and explain in detail operations on queue.

Sol:- Queue is a data structure used for sharing and managing data in a order.

⇒ operations on queue data structure.

1) Enqueue(): It inserts the element at the rear and end of the queue and increments the rear pointer by 1 upon inserting.

2) Dequeue(): It deletes or removes and returns the elements from the front end of the queue and front pointer now. Points to the next element in the queue.

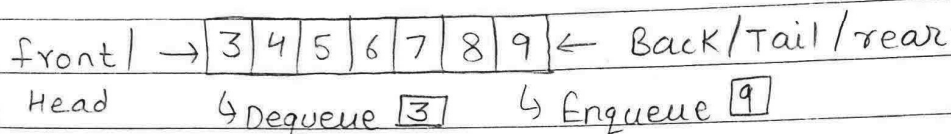
3) Peek(): returns the element at the front of the queue without removing it from queue.

4) Empty(): checks if the queue is empty and return value in true or false.

5) Full(): checks if the queue is full.

Q.2) Explain with neat diagram queue data structure.

Solⁿ:- Queue follows the FIFO (first in first out) where the first element added to the queue is the first one to be removed.



⇒ If we enqueue (9) into the stack, it would be added to the rear end and when we dequeue (), the element at the front pointer will get removed and then displayed onto the screen.

Q.3) Enlist and explain applications and types of queue data structure.

Solⁿ:- Applications of Queue:-

- 1) Task Scheduling:- In operating system queue are used to schedule the tasks in system.
- 2) Data transfer:- Used for data transfer in the network communications.
- 3) Simulation:- In real-world applications to implement waiting lines with queue.
- 4) Priority Queue:- Used for event processing by event processing queues.

Q.4) Write an algorithm of enqueue () and dequeue () operations on simple queue

Solⁿ:-



Enqueue():

- 1) check if queue is full.
- 2) if queue is full, return overflow error and exit.
- 3) if not full, increment the rear pointer to point next empty.
- 4) Add the data element to the queue location where the rear is pointing.
- 5) return success.

Dequeue():

- 1) check queue if empty.
- 2) if queue is empty, return underflow error and exit.
- 3) if the queue is not empty, access the data where front is pointing.
- 4) Increment the front pointer to point the next available data element.
- 5) return success.

Q.5) write an algorithm of enqueue() and dequeue() operations on circular queue.

Solⁿ:- enqueue():

- 1) check if queue is full and if true, return overflow
- 2) if queue is not full:
if queue is empty, Set front, rear = 0
if rear == size - 1, Set rear = 0
Else increment rear++
- 3) Insert the data element at queue [rear].
- 4) return success.



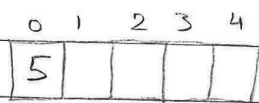
dequeue():

- 1) check if queue is empty, if true return underflow error and exit.
- 2) If queue is not empty, retrieve element at queue [front]
- 3) If $\text{front} == \text{rear}$ (only one element left) set $\text{front} = -1$ and $\text{rear} = -1$.
- 4) Else if $\text{front} == \text{size} - 1$, set $\text{front} = 0$ (wrap around).
- 5) Else increment $\text{front} = \text{front} + 1$
- 6) Return Success.

Q.6) Simulate operations for circular queue:-

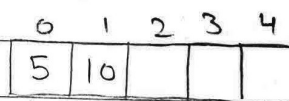
Soln:-

enqueue (5)



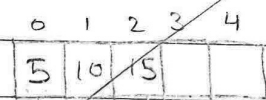
front = rear = index()

enqueue (10)



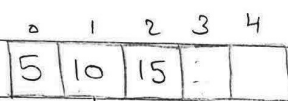
front rear

enqueue (15)



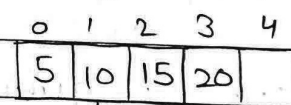
front rear

dequeue ()



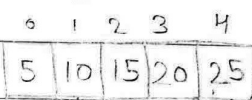
front rear

enqueue (20)



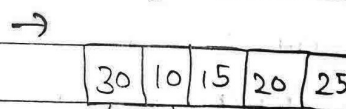
front rear

enqueue (25)



front rear

enqueue (30)



rear front

⇒ rear gets wrapped around from 4 to 0 and it is called as Circular queue.

Conclusion:

AL
15/11/15